SWC3

Software-Entwicklung mit C und C++

Erik Pitzer

WS 2026/27

FH OÖ - Campus Hagenberg - Software Engineering



16. C++ Anhang

Anhang A – C++ Hilfsfunktionen (i)

```
#include <iostream>
#include <format>
#include <print>
#include <fstream>
```

```
#include <cstdint>
#include <stdfloat>
#include <cassert>
#include <cmath>
```

```
#include <string>
#include <string_view>
#include <sstream>
```

```
#include <vector>
#include <map>
#include <set>
#include <unordered_set>
#include <unordered_map>
```

Anhang A – C++ Hilfsfunktionen (ii)

```
#include <iterator>
#include <algorithm>
#include <random>
```

```
#include <memory>
#include <utility>
#include <thread>
```

```
#include <chrono>
#include <regex>
```

```
#include <optional>
#include <expected>
```

Anhang A – C++ Hilfsfunktionen (iii)

```cpp
template<typename T> void show_value(T value) {
  std::print("{}", value);
}

template<std::convertible_to<std::string_view> T>
void show_value(T value) {
  std::print("\"{}\"", value);
}

template<std::convertible_to<std::exception> T>
void show_value(T x) {
  std::print("exception: \"{}\"", x.what());
}

template<>
void show_value(char value)  {
  std::print("'{}'", value);
}
```

Anhang A – C++ Hilfsfunktionen (iv)

```cpp
template<typename T> concept streamable = requires(T value) {
  { std::cout << value } -> std::convertible_to<std::ostream>;
};

template<typename T> requires (std::is_pointer_v<T> && (!std::convertible_to<T, std::string_view>))
void show_value(T pointer) {
  std::print("{:#x}", reinterpret_cast<long>(pointer));
}

template<streamable T>
void show_value(T value) {
  std::cout << value;
}
```

```cpp
template<typename T> concept showable = requires (T value) { show_value(value); };

template<showable T>
void show_value(std::optional<T> value) {
  if (value.has_value()) {
    std::print("Some({})", *value);
  } else {
    std::print("nullopt");
  }
}
```

Anhang A – C++ Hilfsfunktionen (v)

```cpp
template<showable E, showable U>
void show_value(std::expected<E, U> value) {
  if (value.has_value()) {
    std::print("expected(");
    show_value(value.value());
    std::print(")");
  } else {
    std::print("unexpected(");
    show_value(value.error());
    std::print(")");
  }
}
```

```cpp
void show([[maybe_unused]] std::string_view str) {
  std::println();
}

auto split_arg_string(std::string_view str) {
  const std::regex rx{" *([^(<,]*(<[^>]*>)?(\\([^)]*\\))?[^,]*),? *"};
  std::match_results<std::string_view::const_iterator> arg;
  std::regex_search(str.cbegin(), str.cend(), arg, rx);
  return std::make_pair<std::string, std::string_view>(
    arg[1].str(), {arg[0].second, str.cend()});
}
```

Anhang A – C++ Hilfsfunktionen (vi)

```cpp
template<typename T, typename... Types>
void show(std::string_view str, T value, Types... args) {
  auto [first, rest] = split_arg_string(str);
  std::print("{} = ", first); show_value(value);
  if (rest.empty()) {
    std::println();
  } else {
    std::print("; ");
    show(rest, args...);
  }
}
```

```cpp
#define SHOW(...) show(#__VA_ARGS__, __VA_ARGS__)
#define NOTE(...) std::print("// "); std::println(__VA_ARGS__)
#define PL(...) std::println(__VA_ARGS__)
#define P(...) std::print(__VA_ARGS__)
```

```cpp
template<typename T>
constexpr std::string_view compiler_type_name() {
  std::string_view name = __PRETTY_FUNCTION__;
  auto start = name.find('=') + 2;
  auto end = name.find_first_of(";", start);
  return name.substr(start, end - start);
}
```

Anhang A – C++ Hilfsfunktionen (vii)

```cpp
void show_type([[maybe_unused]] std::string_view str) {
  std::println();
}

template <typename T>
void show_type(T&& value) {
  show_value(value);
  std::string name {compiler_type_name<T>()};
  if constexpr (std::is_rvalue_reference_v<T&&>) { name += "&&"; }
  std::print(" ({})", name);
}
```

Anhang A – C++ Hilfsfunktionen (viii)

```cpp
void show_type(std::string_view str, auto&& value, auto&&... args) {
  auto [first, rest] = split_arg_string(str);
  std::print("{} = ", first);
  show_type(std::forward<decltype(value)>(value));
  if (rest.empty()) {
    std::println();
  } else {
    std::print("; ");
    show_type(rest, std::forward<decltype(args)>(args)...);
  }
}

#define SHOW_TYPE(...) show_type(#__VA_ARGS__, __VA_ARGS__)
```

Anhang B – C++ Compiler & -Switches

```
g++ -std=c++23 -pedantic \
 -Wall -Wextra -Wconversion -Wshadow \
 -ftrivial-auto-var-init=pattern -fsanitize=address,undefined,leak  \
 -lasan -lubsan -lm -ldfp \
 -o example.exe example.c
```